PROGRAM 1.A

AIM: Develop a JAVA program to add TWO matrices of suitable order N (The value of N should be read from command line arguments).

OBJECTIVE:

- To accept the matrix order **N** through command line arguments.
- To read elements of two matrices of order **N × N** from the user.
- To perform addition of corresponding elements of the two matrices.
- To display the resultant matrix after addition.

THEORY:

Matrix addition is a basic operation in programming and mathematics.

- Two matrices can be added only if they have the same order (rows and columns).
- The addition is performed by adding corresponding elements of both matrices.
- In Java, command line arguments allow values to be passed during program execution using args[].
- The matrix order **N** is taken from the command line, and elements are read using the Scanner class.

ALGORITHM:

1. Start
2. Check whether at least one command line argument is provided. If NO, display:
   "Please provide the matrix order N as a command line argument."
   and terminate the program.
3. Read the first command line argument and convert it into an integer N.
4. Declare three two-dimensional integer arrays A, B, and C of order N × N.
5. Create a Scanner object to read input from the user.
6. Display message: "Enter elements of first matrix".
7. Read N × N elements from the user and store them in matrix A.
8. Display message: "Enter elements of second matrix".
9. Read N × N elements from the user and store them in matrix B.
10. Add corresponding elements of matrices A and B and store the result in matrix C.
11. Display message: "Resultant Matrix after Addition".
12. Display all elements of matrix C in matrix form.
13. End

FLOWCHART:

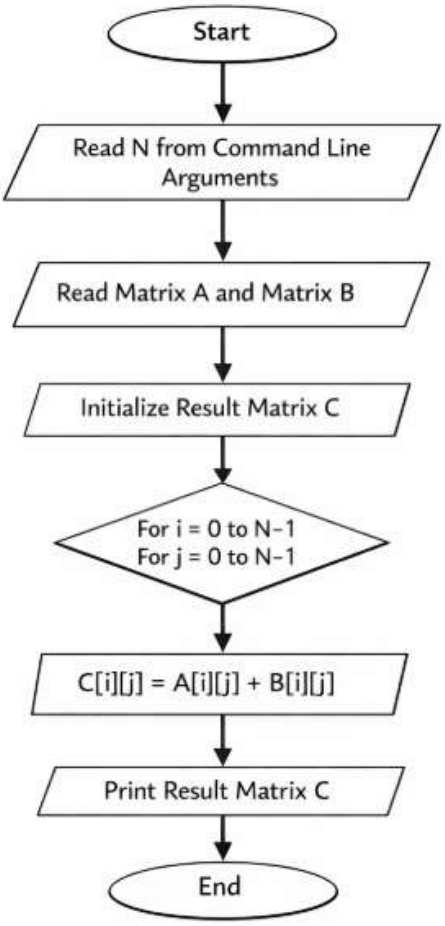


Figure 1-Addition of two matrices

CODE:

```
import java.util.Scanner;
public class MatrixAddition {
 public static void main(String[] args) {
  if (args.length < 1) {
  System.out.println("Please provide the matrix order N as a command line argument.");
   return;
   }
  int N = Integer.parseInt(args[0]);
  int[][] A = new int[N][N];
  int[][] B = new int[N][N];
  int[][] C = new int[N][N];
 Scanner sc = new Scanner(System.in);
System.out.println("Enter elements of first matrix:");
```

```
 for (int i = 0; i < N; i++) {
 for (int j = 0; j < N; j++) {
 A[i][j] = sc.nextInt();
 } }
System.out.println("Enter elements of second matrix:");
for (int i = 0; i < N; i++) {
  for (int j = 0; j < N; j++) {
   B[i][j] = sc.nextInt();
   }  }
for (int i = 0; i < N; i++) {
for (int j = 0; j < N; j++) {
 C[i][j] = A[i][j] + B[i][j];
 }  }
  System.out.println("Resultant Matrix after Addition:");
 for (int i = 0; i < N; i++) {
 for (int j = 0; j < N; j++) {
System.out.print(C[i][j] + " ");
   }
 System.out.println();
 }
 sc.close();
 }}
```

OUTPUT:

PROGRAM-1.B

AIM: Develop a java program to find transpose of matrix of order N using command line input.

ALGORITHM:

1. Start the program.
2. Check whether the command-line argument N is provided; if not, display a message and stop.
3. Read N and create two matrices A[N][N] and T[N][N].
4. Input all elements of matrix A from the user.
5. For each element, assign T[j][i] = A[i][j] to compute the transpose.
6. Display the transpose matrix T.
7. End the program.

FLOWCHART:

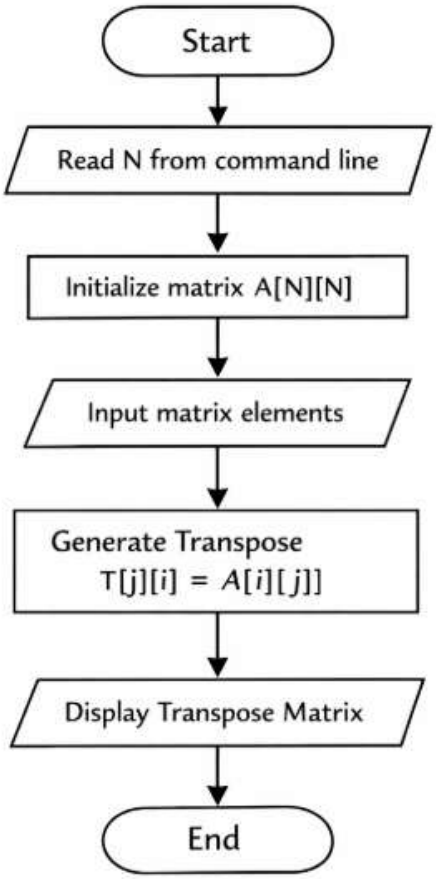


Figure-Transpose of matrix

CODE:

```
import java.util.Scanner;
```

```
public class MatrixTranspose {
   public static void main(String[] args) {
      if (args.length < 1) {
         System.out.println("Please provide matrix order N as command line argument.");
         return;
      }
      int N = Integer.parseInt(args[0]);
      int[][] A = new int[N][N];
      int[][] T = new int[N][N];
      Scanner sc = new Scanner(System.in);
      // Input matrix
      System.out.println("Enter elements of matrix:");
      for (int i = 0; i < N; i++) {
         for (int j = 0; j < N; j++) {
            A[i][j] = sc.nextInt();
         }  }
      // Find transpose
      for (int i = 0; i < N; i++) {
         for (int j = 0; j < N; j++) {
            T[j][i] = A[i][j];
         }  }
  // Display transpose
      System.out.println("Transpose of the matrix:");
      for (int i = 0; i < N; i++) {
         for (int j = 0; j < N; j++) {
            System.out.print(T[i][j] + " ");
         }
         System.out.println();
      }

      sc.close();
```

```
} }
```

OUTPUT:

```
C:\Users\shrut\OneDrive\Desktop>cd C:\Users\shrut\OneDrive\Desktop

C:\Users\shrut\OneDrive\Desktop>javac MatrixTranspose.java

C:\Users\shrut\OneDrive\Desktop>java MatrixTranspose 3
Enter elements of matrix:
1 2 3
2 4 6
4 5 7
Transpose of the matrix:
1 2 4
2 4 5
3 6 7

C:\Users\shrut\OneDrive\Desktop>
```

PROGRAM-2.A

AIM: To develop a Java program that implements a Stack class capable of holding a maximum of 10 integers and to illustrate basic stack operations using a main method.

OBJECTIVE:

- To design a Stack class with a fixed size of 10 elements.
- To implement suitable stack methods such as push, pop, and display.
- To handle stack overflow and underflow conditions.
- To demonstrate stack operations through a Java main method.

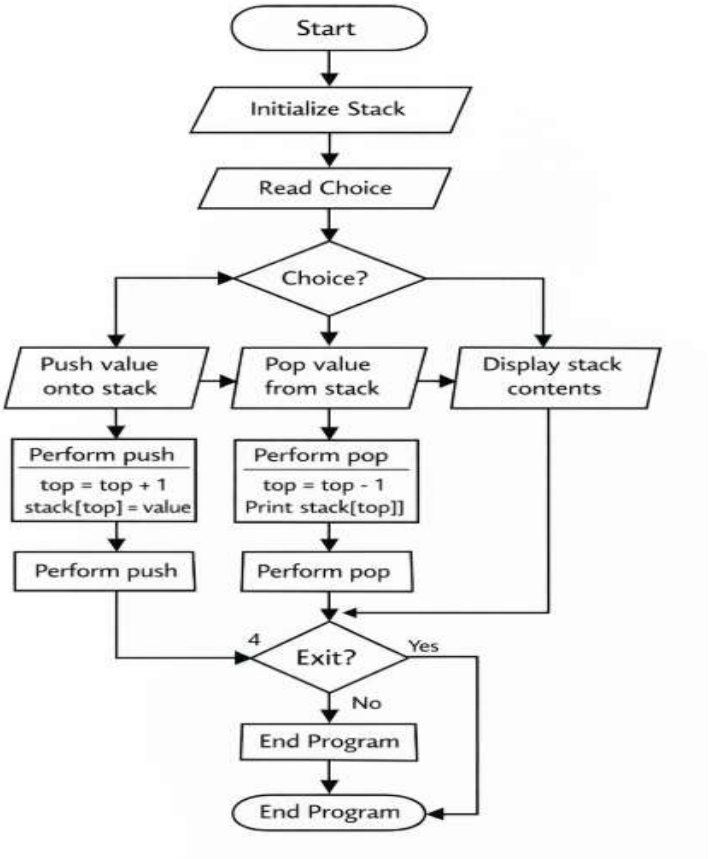
THEORY:

- Push operation inserts an element at the top of the stack.
- Pop operation removes the topmost element from the stack.
- Overflow occurs when an element is pushed into a full stack.
- Underflow occurs when an element is popped from an empty stack.

ALGORITHM:

1. Start
2. Initialize stack array of size 10 and set top = -1.
3. Display menu (Push, Pop, Display, Exit).
4. Read user choice.
5. If choice is Push:
   •Check overflow.
   •Insert element and increment top.
6. If choice is Pop:
   •Check underflow.
   •Remove element and decrement top.
7. If choice is Display:
   •Display all stack elements from top to bottom.
8. If choice is Exit, terminate the program.
9. Repeat steps 3–8 until exit is selected.
10. End

FLOWCHART:



CODE:

```
import java.util.Scanner;
class Stack {
    int[] stack;
    int top;
    int maxSize = 10;
    Stack() {
        stack = new int[maxSize];
        top = -1;
    }
    void push(int value) {
        if (top == maxSize - 1) {
            System.out.println("Stack Overflow! Cannot push element.");
        } else {
            stack[++top] = value;
```

```
            System.out.println("Pushed: " + value);
        } }
    void pop() {
        if (top == -1) {
            System.out.println("Stack Underflow! Cannot pop element.");
        } else {
            System.out.println("Popped: " + stack[top--]);
        }  }
    void display() {
        if (top == -1) {
            System.out.println("Stack is empty.");
        } else {
            System.out.print("Stack elements: ");
            for (int i = top; i >= 0; i--) {
                System.out.print(stack[i] + " ");
            }
            System.out.println();
        } }}
public class StackDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Stack s = new Stack();
        int choice, value;
        do {
            System.out.println("\n1. Push");
            System.out.println("2. Pop");
            System.out.println("3. Display");
            System.out.println("4. Exit");
            System.out.print("Enter your choice: ");
            choice = sc.nextInt();
            switch (choice) {
```

```
            case 1:
                System.out.print("Enter value to push: ");
                value = sc.nextInt();
                s.push(value);
                break;
            case 2:
                s.pop();
                break;
            case 3:
                s.display();
                break;
            case 4:
                System.out.println("Exiting program.");
                break;
            default:
                System.out.println("Invalid choice.");
        } }
while (choice != 4);
    sc.close();
  }}
```

OUTPUT:

```
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 1
Enter value to push: 6
Pushed: 6

1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 3
Stack elements: 6 4 2

1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 2
Popped: 6

1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 4
Exiting program.
PS C:\Users\KHUSHI KAUR\OneDrive\Documents>
```

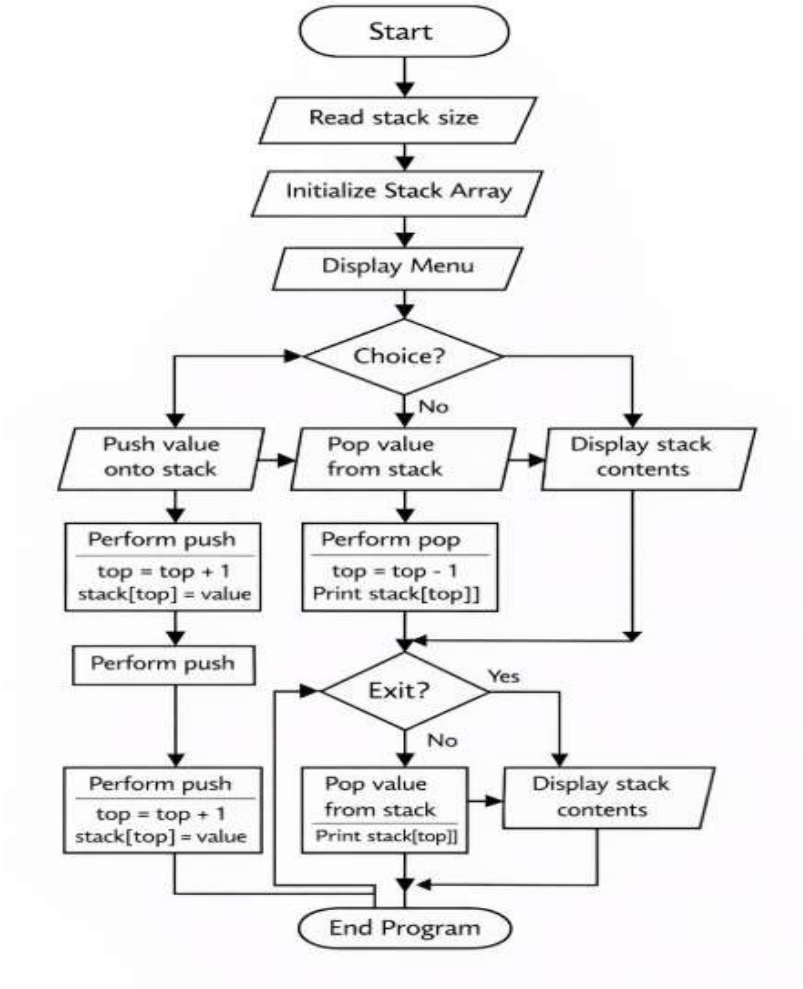
PROGRAM-2.B

AIM: Develop a JAVA program to implement stack using array and demonstrate PUSH, POP, and Display operations.

ALGORITHM:

1. Start the program.
2. Create a stack array of given size and initialize top = -1.
3. Display menu: PUSH, POP, DISPLAY, EXIT.
4. Read user's choice.
5. If choice is PUSH, check overflow (top == size-1); if not full, insert element and increment top.
6. If choice is POP, check underflow (top == -1); if not empty, remove and display top element, then decrement top.
7. If choice is DISPLAY, print all stack elements from top to bottom.
8. Repeat steps 3–7 until EXIT is chosen.
9. End the program.

FLOWCHART:



CODE:

```
import java.util.Scanner;
class StackArray {
    int[] stack;
    int top;
    int maxSize;
    // Constructor
    StackArray(int size) {
        maxSize = size;
        stack = new int[maxSize];
        top = -1;
    }
    // PUSH operation
    void push(int value) {
        if (top == maxSize - 1) {
            System.out.println("Stack Overflow! Cannot push.");
        } else {
            stack[++top] = value;
            System.out.println("Pushed: " + value);
        }  }
    // POP operation
    void pop() {
        if (top == -1) {
            System.out.println("Stack Underflow! Cannot pop.");
        } else {
            System.out.println("Popped: " + stack[top--]);
        }  }
    // DISPLAY operation
    void display() {
```

```
        if (top == -1) {
            System.out.println("Stack is empty.");
        } else
{
  System.out.println("Stack elements:");
            for (int i = top; i >= 0; i--) {
                System.out.println(stack[i]);
            } }  }}
public class StackDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter stack size: ");
        int size = sc.nextInt();
   StackArray stack = new StackArray(size);
   while (true) {
  System.out.println("\n--- Stack Menu ---");
   System.out.println("1. Push");
System.out.println("2. Pop");
 System.out.println("3. Display");
 System.out.println("4. Exit");

   System.out.print("Enter your choice: ");
   int choice = sc.nextInt();
   switch (choice) {
   case 1:
 System.out.print("Enter value to push: ");
 int val = sc.nextInt();
 stack.push(val);
 break;
case 2:
stack.pop();
```

```
 break;
 case 3:
stack.display();
 break;
  case 4:
  System.out.println("Exiting program.");
 sc.close();
  return;
 default:
 System.out.println("Invalid choice.");
 }}}}
```

OUTPUT:

```
C:\Users\shrut\OneDrive\Desktop>java StackDemo
Enter stack size: 4

--- Stack Menu ---
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 1
Enter value to push: 6
Pushed: 6

--- Stack Menu ---
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 2
Popped: 6

--- Stack Menu ---
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 3
Stack is empty.
```

PROGRAM-3.A

AIM: To develop a Java program that defines an Employee class with attributes ID, name, and salary, and to demonstrate salary increment using a suitable main method.

OBJECTIVE:

- To design an Employee class with data members: employee ID, name, and salary.
- To implement a method to raise the salary by a given percentage.
- To create objects of the Employee class and demonstrate its functionality.
- To display employee details before and after salary increment.

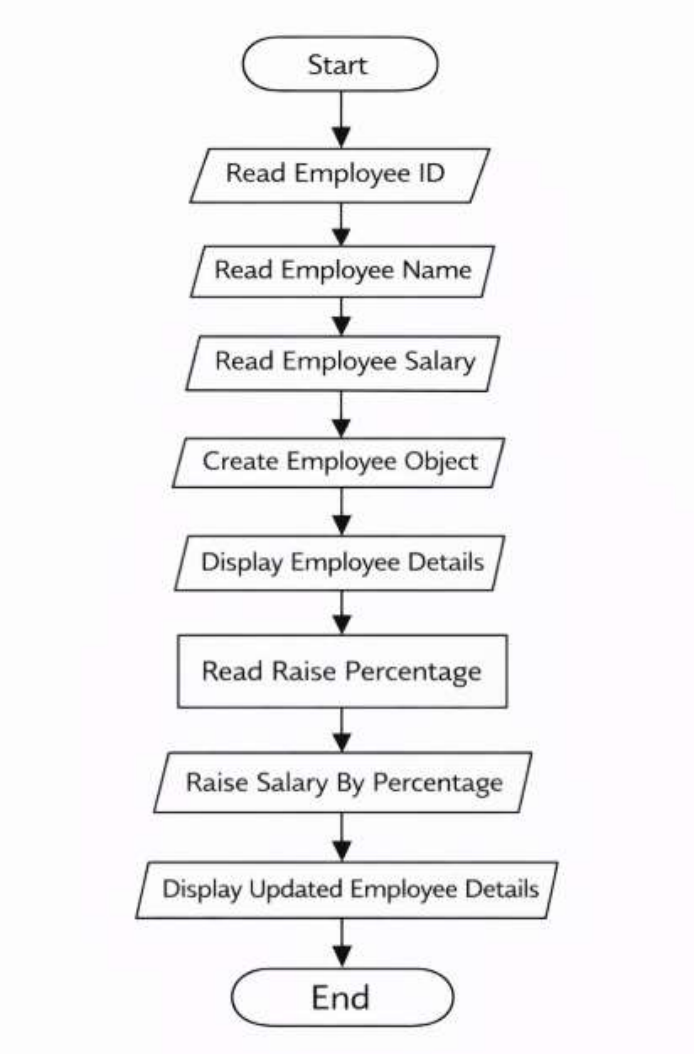
THEORY:

In Object-Oriented Programming (OOP), a class represents a blueprint for creating objects.
- Encapsulation allows bundling data and methods within a class.
- Methods operate on object data to perform specific tasks.
- The salary raise operation updates the salary based on a given percentage using a mathematical formula.

ALGORITHM:

1. Start the program.
2. Create an Employee class with data members: id, name, and salary.
3. Define and initialize employee id, name, and salary
4. Define a method raiseSalary(percent) to calculate the updated salary using the given percentage.
5. Define a method display() to print employee details.
6. In the main method, create a Scanner object to take input from the user.
7. Accept employee id, name, and salary from the user.
8. Create an Employee object using the entered details.
9. Display employee details before salary increment.
10. Step 10: Accept the percentage value for salary increment.
11. Step 11: Call raiseSalary() method to update the employee salary.
12. Step 12: Display employee details after salary increment.
13. Step 13: Close the Scanner object.
14. Step 14: End the program.

FLOWCHART:



CODE:

```
import java.util.Scanner;
class Employee {
    private int id;
    private String name;
    private double salary;
    Employee(int empid, String empname, double empsalary) {
        id = empid;
        name = empname;
        salary = empsalary;
    }
    void raiseSalary(double percent) {
        salary = salary + (salary * percent / 100);
    }
    void display() {
        System.out.println("Employee ID   : " + id);
        System.out.println("Employee Name : " + name);
        System.out.println("Salary        : " + salary);
```

```
    }
}
public class EmployeeDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter Employee ID: ");
        int id = sc.nextInt();
        sc.nextLine();
        System.out.print("Enter Employee Name: ");
        String name = sc.nextLine();
        System.out.print("Enter Employee Salary: ");
        double salary = sc.nextDouble();
        Employee emp = new Employee(id, name, salary);
        System.out.println("\nEmployee Details Before Salary Raise:");
        emp.display();
        System.out.print("\nEnter percentage to raise salary: ");
        double percent = sc.nextDouble();
        emp.raiseSalary(percent);
        System.out.println("\nEmployee Details After Salary Raise:");
        emp.display();
        sc.close();
    }
}
```

OUTPUT:

```
Enter Employee ID: 101
Enter Employee Name: Khushi Kaur
Enter Employee Salary: 53000

Employee Details Before Salary Raise:
Employee ID   : 101
Employee Name : Khushi Kaur
Salary        : 53000.0

Enter percentage to raise salary: 15

Employee Details After Salary Raise:
Employee ID   : 101
Employee Name : Khushi Kaur
Salary        : 60950.0
```

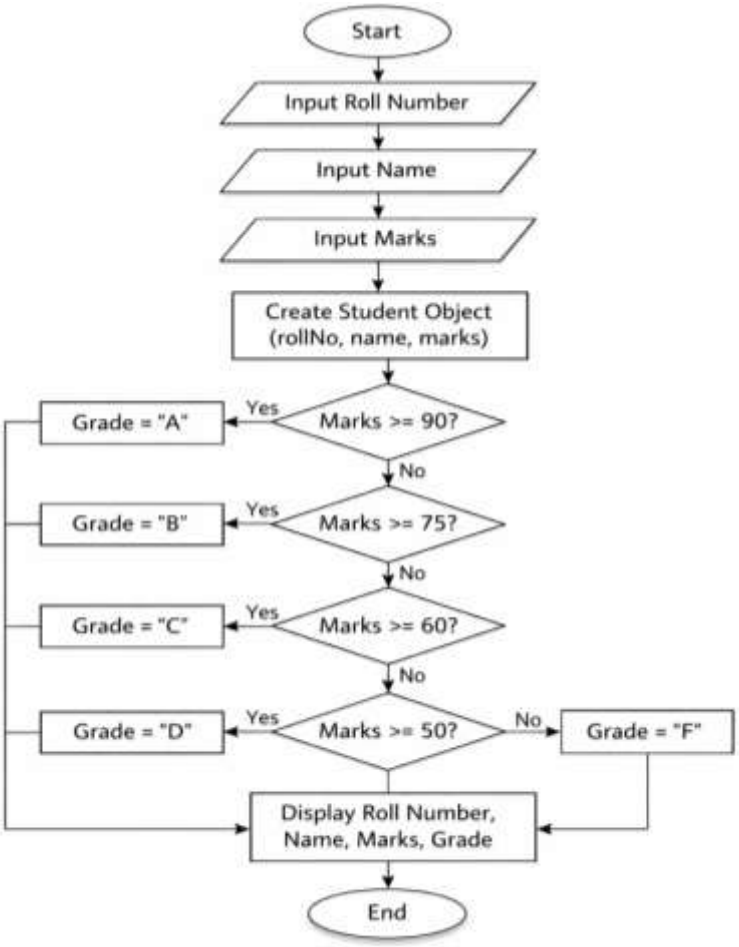
PROGRAM 3-B

AIM: Develop a JAVA program to create a Student class with roll number, name, marks and method to calculate grade.

ALGORITHM:

1. Start
2. Read roll number
3. Read name
4. Read marks
5. Create Student object
6. If marks ≥ 90 → Grade = A
7. Else if marks ≥ 75 → Grade = B
8. Else if marks ≥ 60 → Grade = C
9. Else if marks ≥ 50 → Grade = D
10. Else → Grade = F
11. Display student details with grade
12. Stop

FLOWCHART:



CODE

```
import java.util.Scanner;
class Student {
    int rollNo;
    String name;
    double marks;
  Student(int r, String n, double m) {
        rollNo = r;
        name = n;
        marks = m;
    }
 String calculateGrade() {
        if (marks >= 90)
            return "A";
        else if (marks >= 75)
            return "B";
        else if (marks >= 60)
            return "C";
        else if (marks >= 50)
            return "D";
        else
            return "F";
    } void display() {
        System.out.println("Roll Number : " + rollNo);
        System.out.println("Name        : " + name);
        System.out.println("Marks       : " + marks);
        System.out.println("Grade       : " + calculateGrade());
    }
}

public class StudentDemo {
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Roll Number: ");
        int roll = sc.nextInt();
        sc.nextLine();

        System.out.print("Enter Name: ");
        String name = sc.nextLine();

        System.out.print("Enter Marks: ");
        double marks = sc.nextDouble();

        Student s = new Student(roll, name, marks);

        System.out.println("\nStudent Details:");
        s.display();

        sc.close();
    }
}
```

OUTPUT:

```
Enter Roll Number: 201
Enter Name: Khushi Kaur
Enter Marks: 96

Student Details:
Roll Number : 201
Name        : Khushi Kaur
Marks       : 96.0
Grade       : A
```

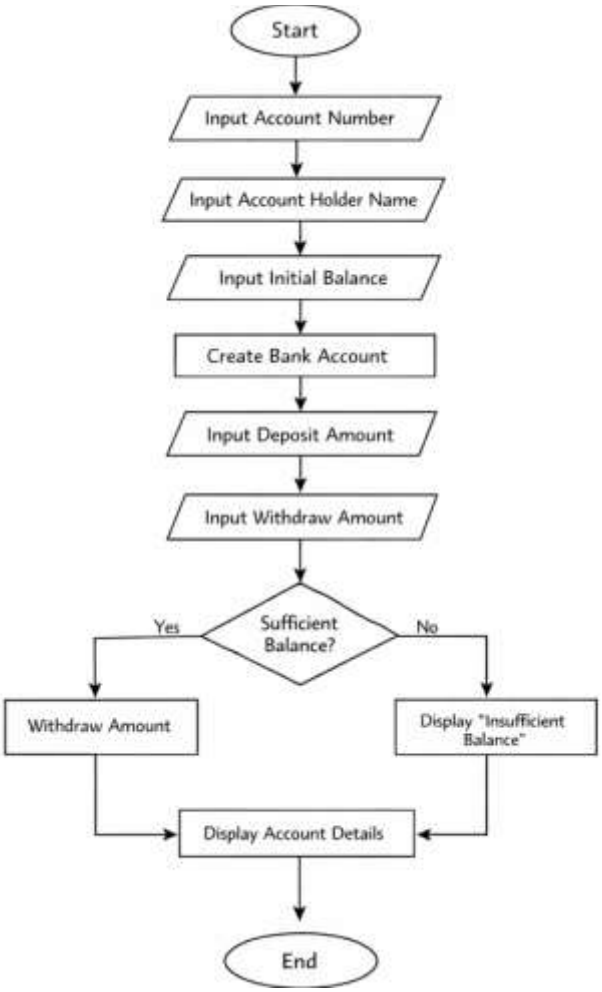
PROGRAM 3C

AIM: Develop a JAVA program to create a BankAccount class with deposit and withdraw methods

ALGORITHM:

1. Start
2. Read account number
3. Read account holder name
4. Read initial balance
5. Create BankAccount object
6. Read deposit amount
7. Add deposit amount to balance
8. Read withdrawal amount
9. If withdrawal amount ≤ balance
   Subtract amount from balance
   Else
   Display "Insufficient Balance"
10. Display account details
11. Stop

FLOWCHART:



CODE:

```
import java.util.Scanner;
```

```
class BankAccount {
    int accountNumber;
    String accountHolder;
    double balance;

    BankAccount(int accNo, String name, double bal) {
        accountNumber = accNo;
        accountHolder = name;
        balance = bal;
    }

    void deposit(double amount) {
        balance = balance + amount;
        System.out.println("Amount Deposited Successfully.");
    }

    void withdraw(double amount) {
        if (amount <= balance) {
            balance = balance - amount;
            System.out.println("Amount Withdrawn Successfully.");
        } else {
            System.out.println("Insufficient Balance.");
        }
    }

    void display() {
        System.out.println("Account Number  : " + accountNumber);
        System.out.println("Account Holder  : " + accountHolder);
        System.out.println("Current Balance : " + balance);
    }
}
```

```
public class BankDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Account Number: ");
        int accNo = sc.nextInt();
        sc.nextLine();

        System.out.print("Enter Account Holder Name: ");
        String name = sc.nextLine();

        System.out.print("Enter Initial Balance: ");
        double balance = sc.nextDouble();

        BankAccount acc = new BankAccount(accNo, name, balance);

        System.out.print("Enter amount to deposit: ");
        double dep = sc.nextDouble();
        acc.deposit(dep);

        System.out.print("Enter amount to withdraw: ");
        double wd = sc.nextDouble();
        acc.withdraw(wd);

        System.out.println("\nAccount Details:");
        acc.display();

        sc.close();
    }
}
```

OUTPUT:

```
Enter Account Number: 10111
Enter Account Holder Name: Khushi Kaur
Enter Initial Balance: 23000
Enter amount to deposit: 34000
Amount Deposited Successfully.
Enter amount to withdraw: 5000
Amount Withdrawn Successfully.

Account Details:
Account Number  : 10111
Account Holder  : Khushi Kaur
Current Balance : 52000.0
```

PROGRAM 4

Aim: To develop a Java program that defines a **MyPoint** class to model a 2D point and to test all its methods using a suitable main program.

Objective:

- To create a class with x and y coordinates representing a point in 2D space.
- To implement constructors for default and parameterized initialization.
- To define methods for setting and getting coordinates.
- To calculate distance between two points using method overloading.
- To demonstrate all class methods using a test program.

Theory:

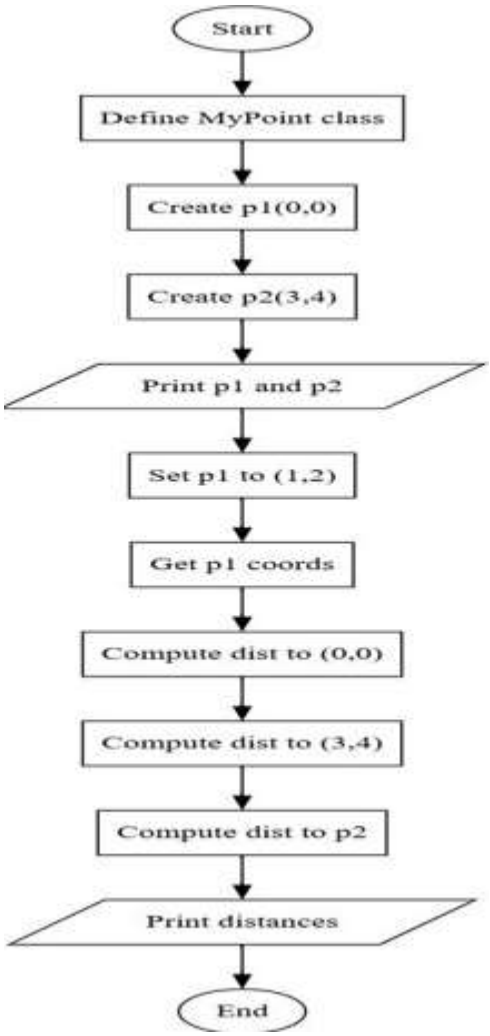
- Constructor overloading allows creating objects in different ways.
- Method overloading allows multiple methods with the same name but different parameters.
- The distance between two points is calculated using the Euclidean distance formula:

Distance = $\sqrt{[(x_2 - x_1)^2 + (y_2 - y_1)^2]}$.

Algorithm:

1. **Start**
2. Declare class MyPoint with variables x and y.
3. Define constructors to initialize coordinates.
4. Define methods:
5. setXY() to set values
6. getXY() to return coordinates
7. distance() methods to calculate distance
    a. From origin
    b. From given point
    c. From another object
8. In main():
9. Create two objects p1 and p2.
10. Display their coordinates.
11. Update p1 using setXY().
12. Calculate and display distances.
13. **Stop**

Flowchart:



Implementation:

```
class MyPoint {
   private int x;
   private int y;
   MyPoint() {
      x = 0;
      y = 0;
   }
   MyPoint(int x, int y) {
      this.x = x;
```

```
        this.y = y;
    }
    void setXY(int x, int y) {
        this.x = x;
        this.y = y;
    }
    int[] getXY() {
        return new int[]{x, y};
    }
    public String toString() {
        return "(" + x + ", " + y + ")";
    }
    double distance(int x, int y) {
        return Math.sqrt((this.x - x) * (this.x - x) + (this.y - y) * (this.y - y));
    }
    double distance(MyPoint another) {
        return Math.sqrt((this.x - another.x) * (this.x - another.x) + (this.y - another.y) * (this.y -
another.y));
    }
    double distance() {
        return Math.sqrt(x * x + y * y);
    }
}

public class TestMyPoint {
    public static void main(String[] args) {
        MyPoint p1 = new MyPoint();
        MyPoint p2 = new MyPoint(3, 4);
        System.out.println("Point p1: " + p1);
```

```
        System.out.println("Point p2: " + p2);
        p1.setXY(1, 2);
        int[] xy = p1.getXY();
        System.out.println("After setXY, p1: (" + xy[0] + ", " + xy[1] + ")");
        System.out.println("Distance from p1 to (0,0): " + p1.distance());
        System.out.println("Distance from p1 to (3,4): " + p1.distance(3, 4));
        System.out.println("Distance from p1 to p2: " + p1.distance(p2));
    }
}
```

OUTPUT:

BlueJ: Terminal Window - programming

Options

```
Point p1: (0, 0)
Point p2: (3, 4)
After setXY, p1: (1, 2)
Distance from p1 to (0,0): 2.23606797749979
Distance from p1 to (3,4): 2.8284271247461903
Distance from p1 to p2: 2.8284271247461903
```

EXPERIMENT-5

Aim: To develop a Java program that demonstrates polymorphism by creating a base class Shape and derived classes Circle, Triangle, and Square, each implementing draw() and erase() methods.

Objective:

- To create a base class named Shape.
- To derive subclasses Circle, Triangle, and Square from Shape.
- To override the draw() and erase() methods in each subclass.
- To demonstrate runtime polymorphism using method overriding and dynamic method dispatch.

Theory:

- Method overriding occurs when a subclass provides a specific implementation of a method already defined in the parent class.
- Runtime polymorphism is achieved using inheritance and method overriding.
- A base class reference can refer to objects of its derived classes, and the appropriate method is called at runtime.

Implementation:

```
class Shape {
   void draw() {
      System.out.println("Drawing Shape");
   }
   void erase() {
      System.out.println("Erasing Shape");
   }
}
class Circle extends Shape {
   void draw() {
      System.out.println("Drawing Circle");
   }
   void erase() {
      System.out.println("Erasing Circle");
   }
}
class Triangle extends Shape {
```

```
    void draw() {
        System.out.println("Drawing Triangle");
    }
    void erase() {
        System.out.println("Erasing Triangle");
    }
}
class Square extends Shape {
    void draw() {
        System.out.println("Drawing Square");
    }
    void erase() {
        System.out.println("Erasing Square");
    }
}
public class ShapeDemo {
    public static void main(String[] args) {
        Shape s;
        s = new Circle();
        s.draw();
        s.erase();
        s = new Triangle();
        s.draw();
        s.erase();
        s = new Square();
        s.draw();
        s.erase();
    }
}
```

Experiment-7

Aim: To develop a Java program that demonstrates interfaces by creating a Resizable interface and implementing it in a Rectangle class.

Algorithm:

Step 1: Start the program.

Step2:Create an interface Resizable with two methods: resizeWidth(int width) and resizeHeight(int height).

Step 3: Create a class Rectangle that implements the Resizable interface.

Step 4: Declare variables width and height in the Rectangle class.

Step 5: Create a constructor to initialize width and height.

Step 6: Implement the methods resizeWidth() and resizeHeight() to change the rectangle dimensions.

Step 7: Create a method display() to print the width and height.

Step 8: In the main() method, create a Rectangle object with initial values.

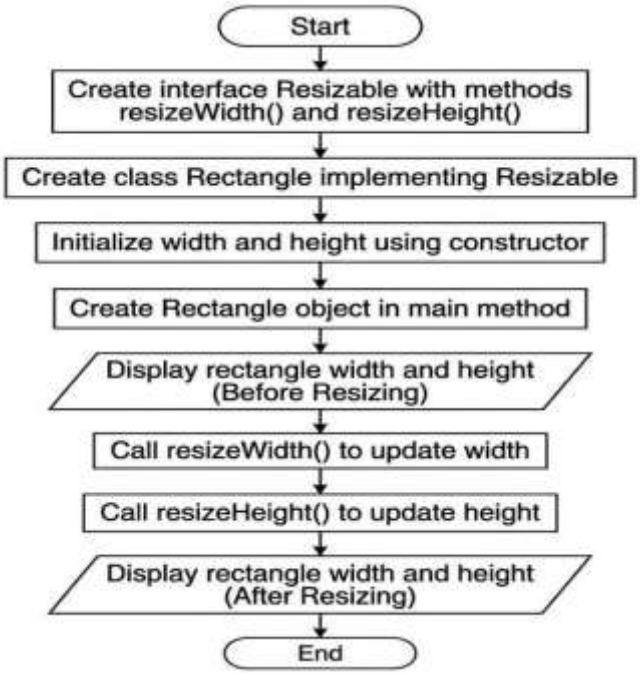
Step 9: Display the rectangle dimensions before resizing.

Step 10: Call resizeWidth() and resizeHeight() to change the dimensions.

Step 11: Display the rectangle dimensions after resizing.

Step 12: End the program.

Flowchart:



Implementation:

```
interface Resizable {
void resizeWidth(int width);
void resizeHeight(int height);
}
class Rectangle implements Resizable {
private int width;
private int height;
Rectangle(int width, int height) {
this.width = width;
this.height = height;
}
public void resizeWidth(int width) {
this.width = width;
}
public void resizeHeight(int height) {
this.height = height;
}
void display() {
System.out.println("Rectangle Width: " + width);
System.out.println("Rectangle Height: " + height);
}
}
public class ResizableDemo {
public static void main(String[] args) {
Rectangle r = new Rectangle(10, 5);
System.out.println("Before Resizing:");
r.display();
r.resizeWidth(20);
r.resizeHeight(10);
System.out.println("After Resizing:");
```

```
r.display();
}
}
```

Output:

```
C:\Users\KHUSHI KAUR\OneDrive\Pictures\Documents>java ResizableDemo
Before Resizing:
Rectangle Width: 10
Rectangle Height: 5
After Resizing:
Rectangle Width: 20
Rectangle Height: 10

C:\Users\KHUSHI KAUR\OneDrive\Pictures\Documents>
```